REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay

ISO 9001: 2015
SOCOTEC SCP000722Q

BAGONG PILIPINAS

WEB SYSTEMS AND TECHNOLOGIES

Phase 5 – Displaying and Managing Data on the Frontend

LAB REPORT

Uniformity: An E-Commerce platform for Pre-Loved School and Professional Uniforms

GROUP NAME: TechNic **Course & Block:** BSIT-2A		
MEMBERS	**ROLE**	**SIGNATURE**
Cipriano, Mcxyron B.	*Back-End Developer*	
Hernandez, Kurt Jushua S.	*Database Manager/Tester & Debugger*	
Registrado, Mharie Franz	*Tester & Debugger*	
Red, Paul Orlando B.	*Project, Database, GitHub Manager & Documentation Officer*	
Romano, Jay L.	*Front-End Developer*	

Submitted to:

Guillermo V. Red, Jr., DIT

Assistant Professor IV

I. Overview

This lab report documents Phase 5 of our Uniformity project — Data Retrieval & Display. Building on the form submission and data insertion work completed in Phase 4, this phase focuses on fetching real data from our MongoDB database via the REST API and dynamically rendering it on the frontend using vanilla JavaScript and Bootstrap 5.

The primary deliverables of this phase include: (1) a fully functional product browsing dashboard that loads and filters live product listings; (2) a product detail page that fetches individual product data by ID; (3) a dynamic cart page that displays real cart items from the backend; (4) live seller listings in My Listings; (5) buyer order history in My Orders; and (6) populated profile statistics.

II. Display Features Implemented

The table below lists all data retrieval and display features implemented in Phase 5:

Display Feature	API Endpoint	Rendered
Dashboard – Browse Uniforms	`GET /api/products`	Product grid with image, name, size, price, seller; category & size filters; live count; pagination
Product Detail Page	`GET /api/products/:id`	Full product info, seller card, quantity selector, Add to Cart button wired to real data
Cart Page	`GET /api/cart/:userId`	Real cart items fetched & rendered; quantity controls; remove item; dynamic total calculation
My Listings (Seller)	`GET /api/products?seller=:id`	Seller's own products in table; live stat cards; incoming order items per seller
My Orders (Buyer)	`GET /api/orders/buyer/:userId`	Order history list with status badges; click-through to order detail
My Order Details	`GET /api/orders/buyer/:id + items`	Per-order items, delivery info, confirm receipt button wired to PATCH /deliver
Profile Page	`GET /api/auth/me + orders + products`	Real name/email/avatar; dynamic stats for orders placed, listings, spent, earned

Display Feature	API Endpoint	Rendered
Category Filter	`GET /api/categories`	Populates filter dropdown on dashboard with real DB categories (already done in Phase 4 main.js)

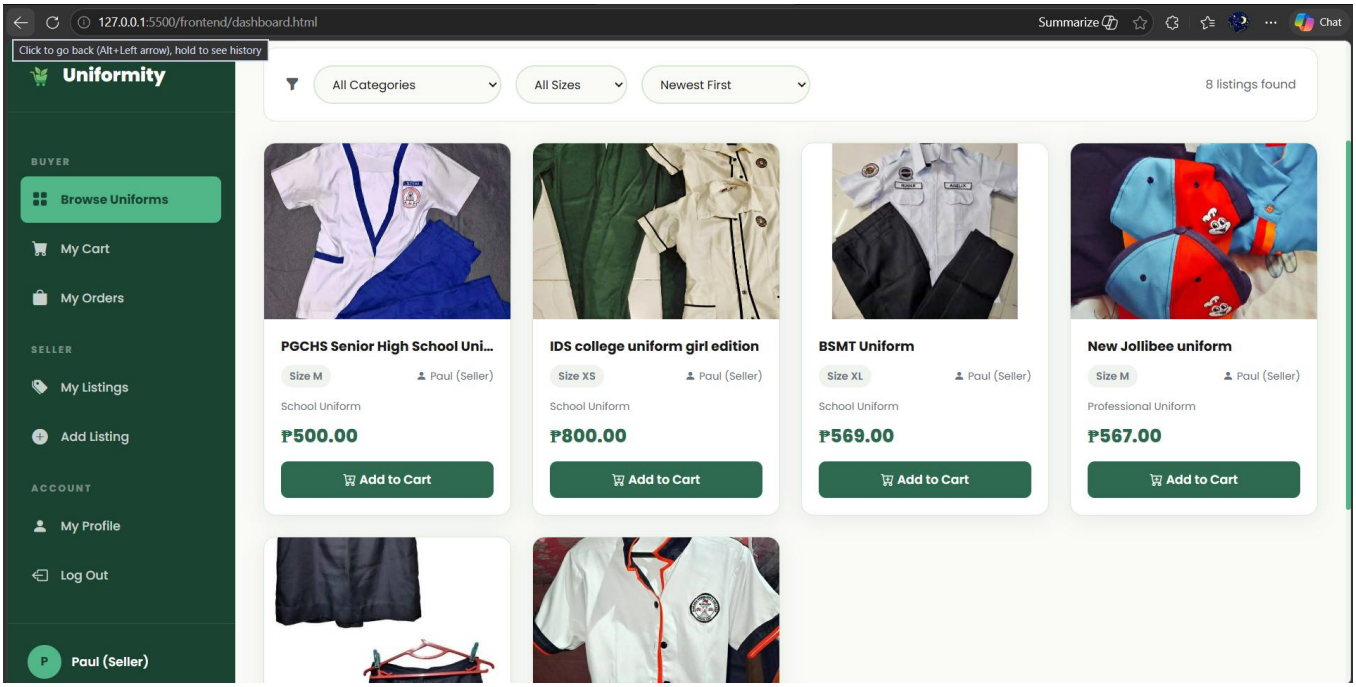
III. Display Page Screenshots

Screenshots below show each data-driven page rendered in the browser, demonstrating real MongoDB data being fetched and displayed dynamically.

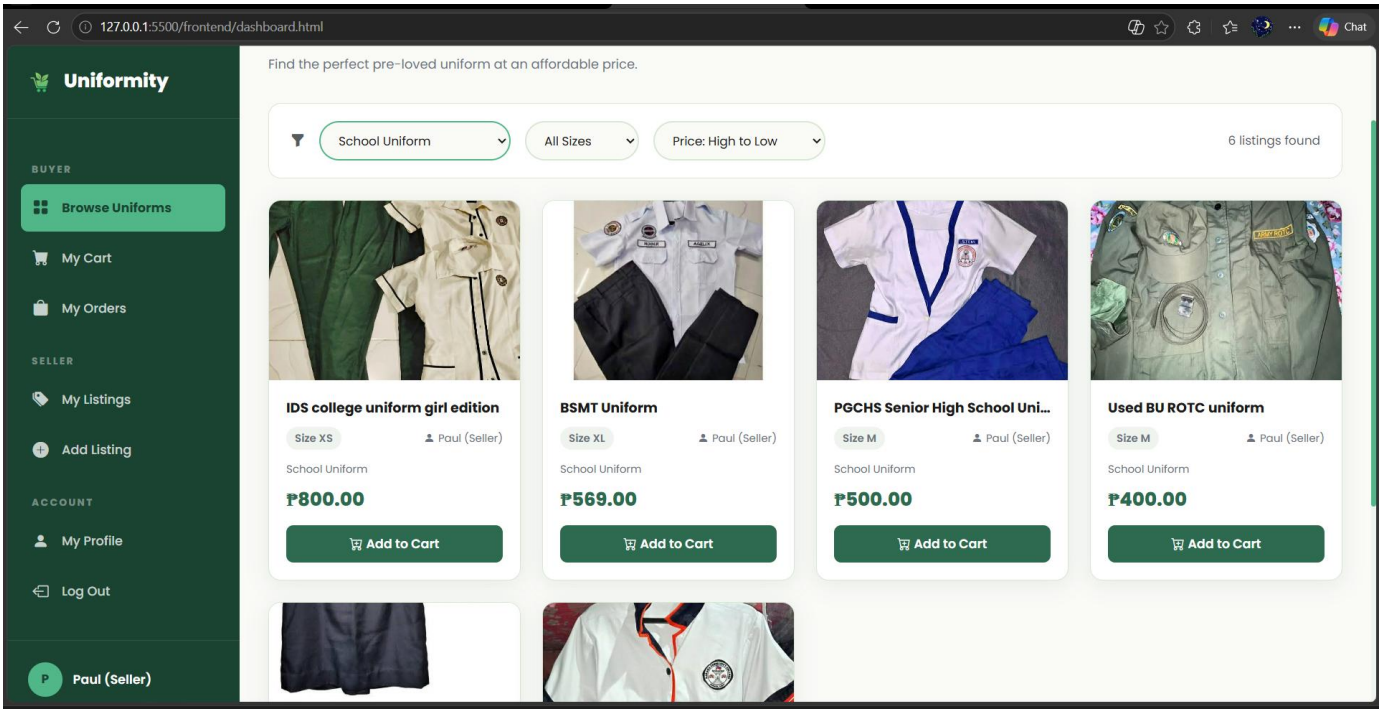
A. Dashboard – Browse Uniforms (dashboard.html)

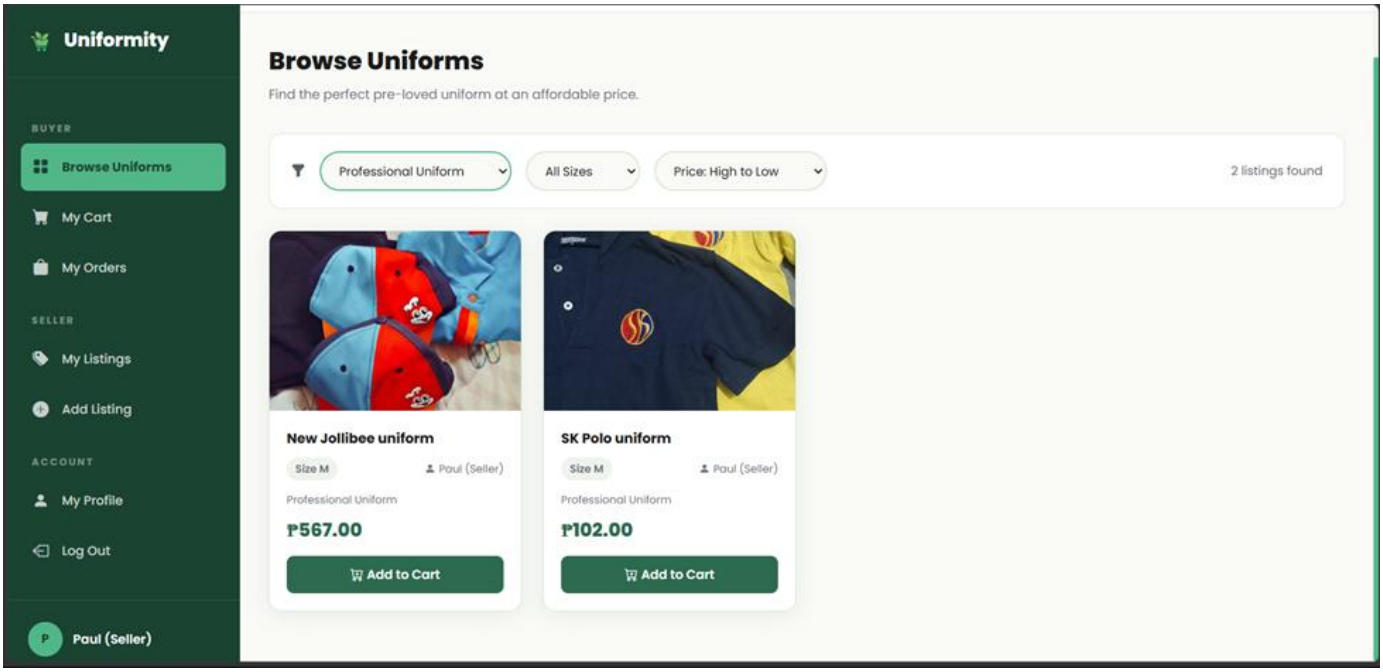
Route: GET /api/products | Shows real product cards populated from the database with category/size filters active.

Product Grid – All Categories — Real product cards rendered from GET /api/products



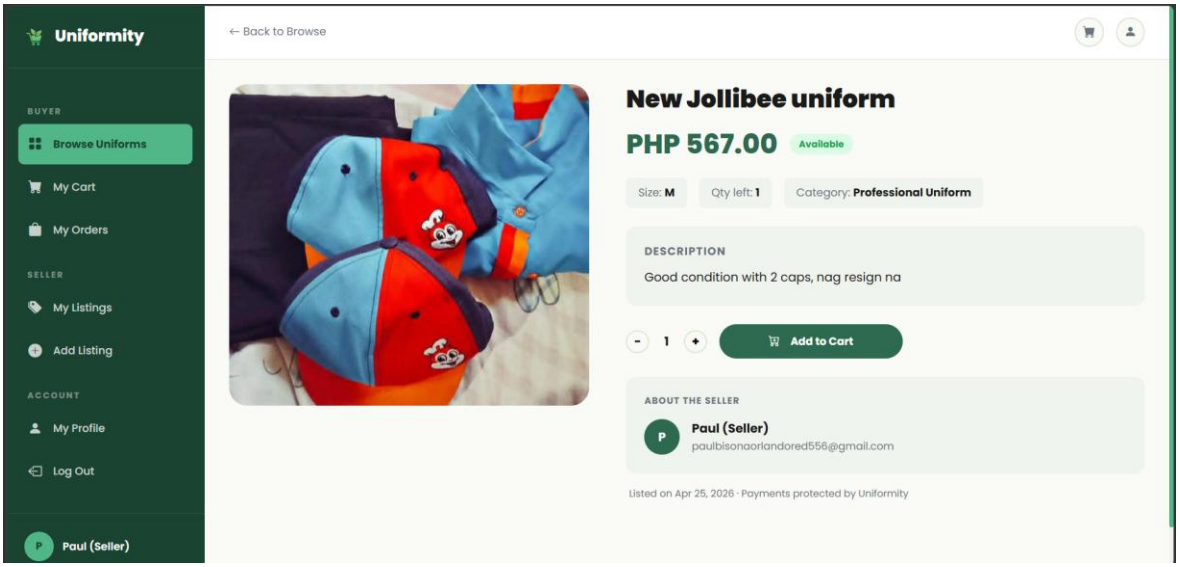
Dashboard – Filtered by Category — Filter dropdown populated from GET /api/categories; grid updates on selection

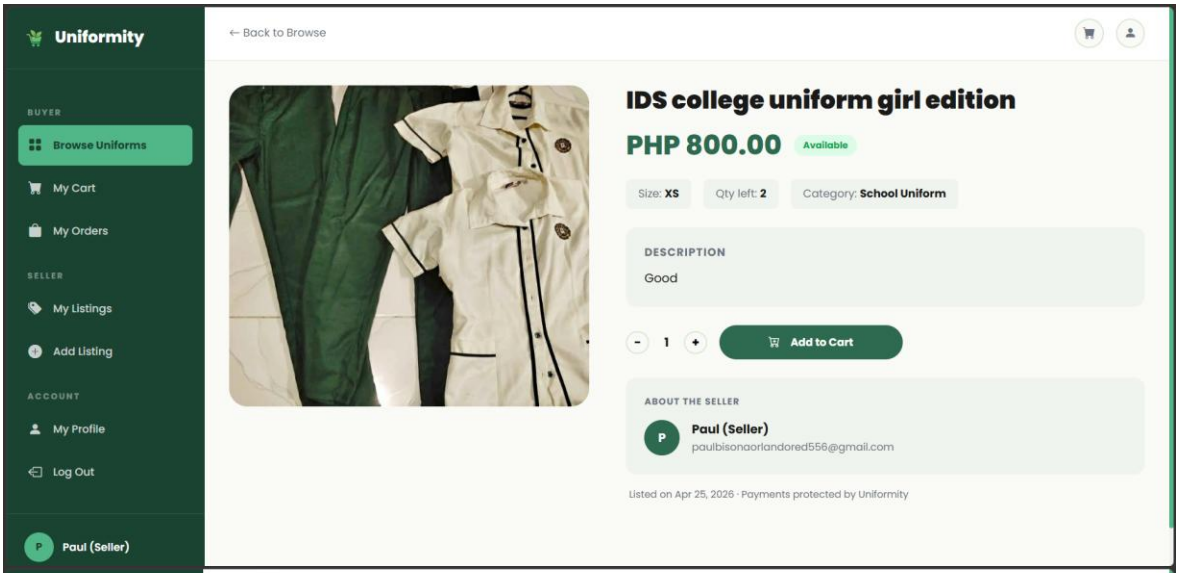


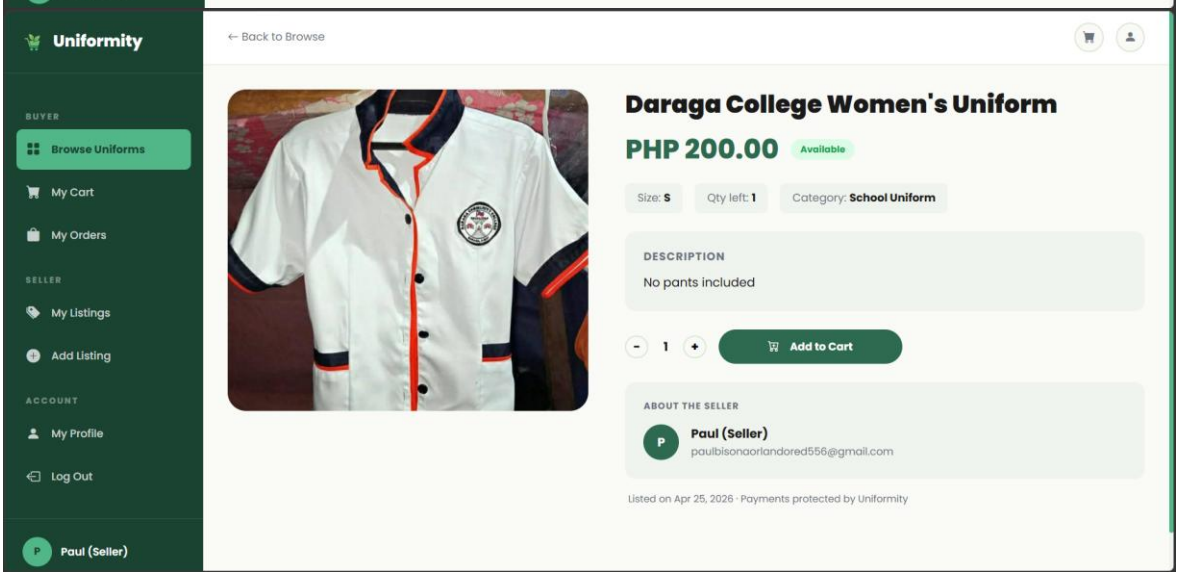


B. Product Detail Page (product-detail.html)

Route: GET /api/products/:id | Reads ?id= from URL, fetches product, renders name, price, size, quantity, description, seller info, and image.

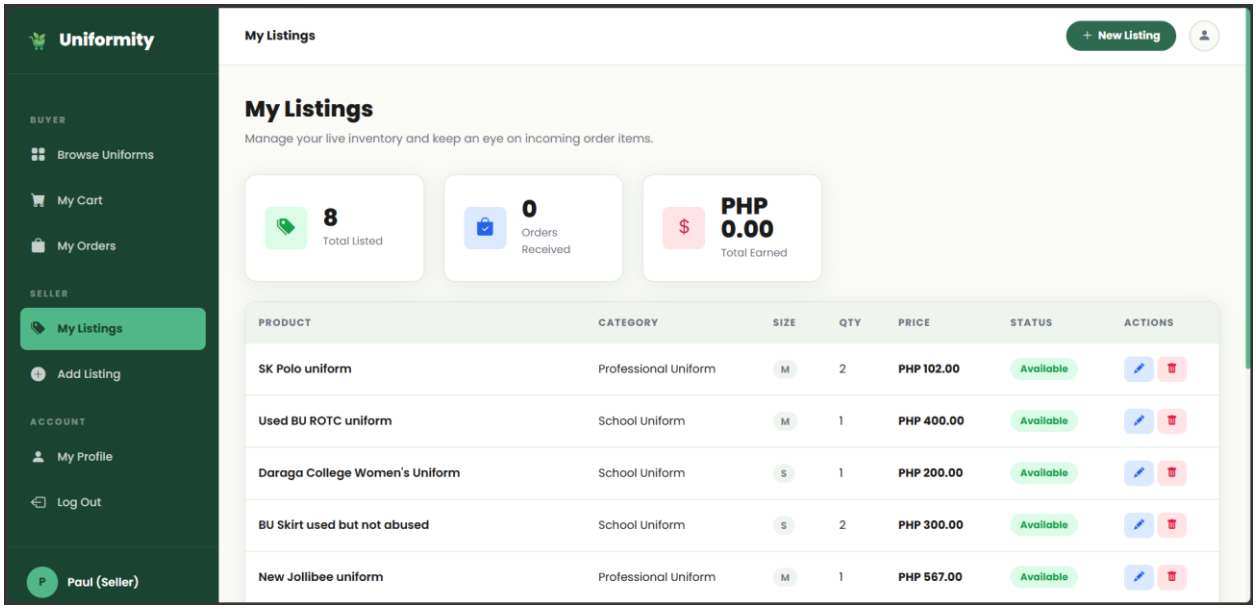






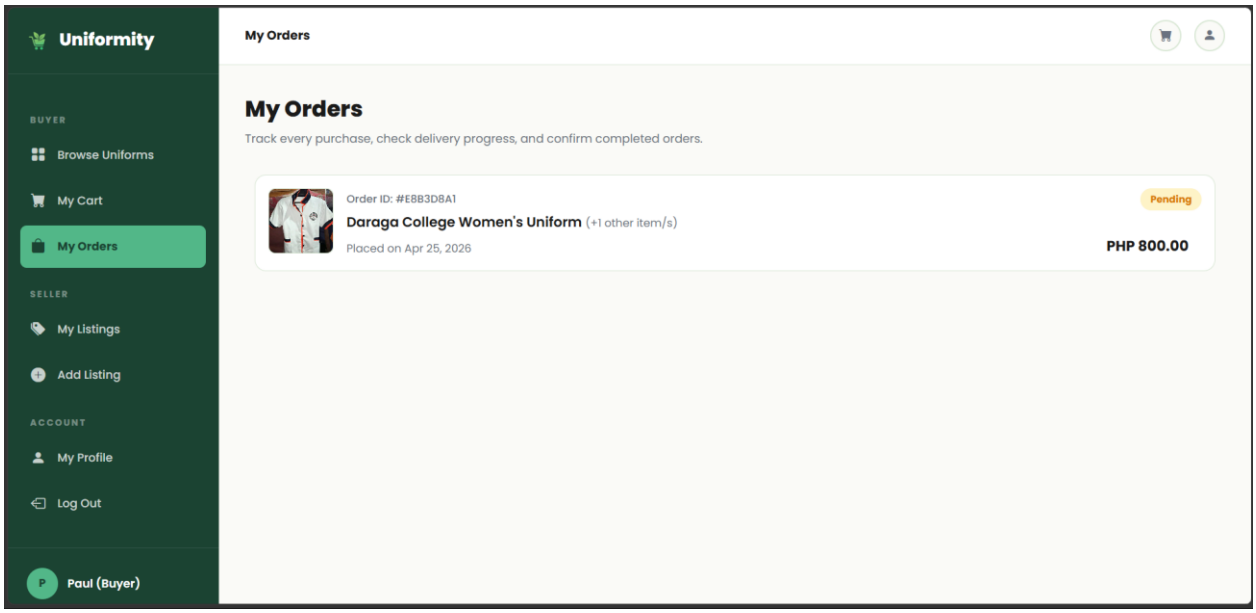
C. My Listings – Seller View (my-listings.html)

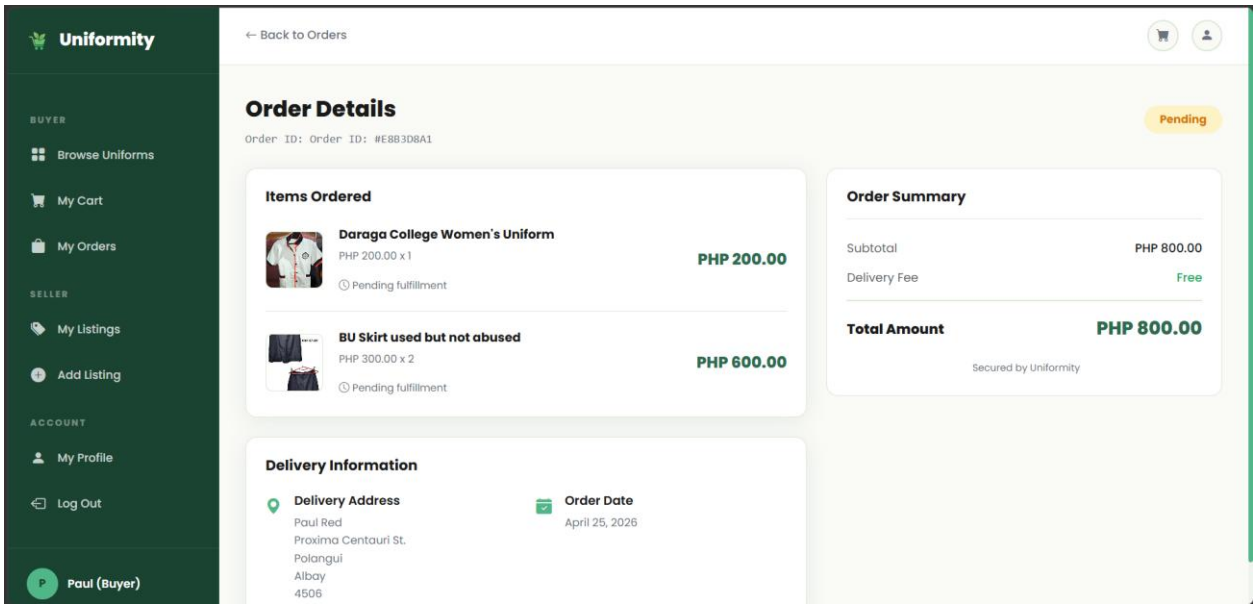
Route: GET /api/products?seller=:id | Displays seller's own product listings in a table with live stat cards.



D. My Orders – Buyer View (my-orders.html)

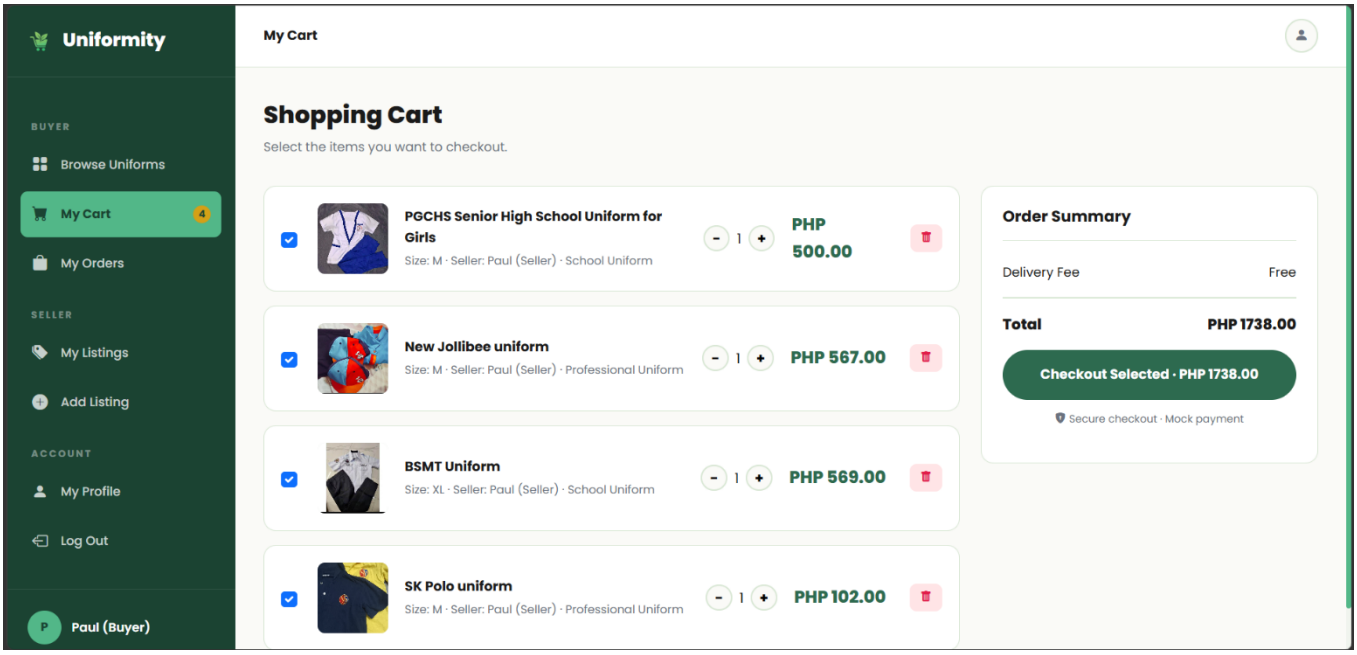
Route: GET /api/orders/buyer/:userId | Lists all buyer orders with status badges and formatted dates.





E. Shopping Cart (cart.html)

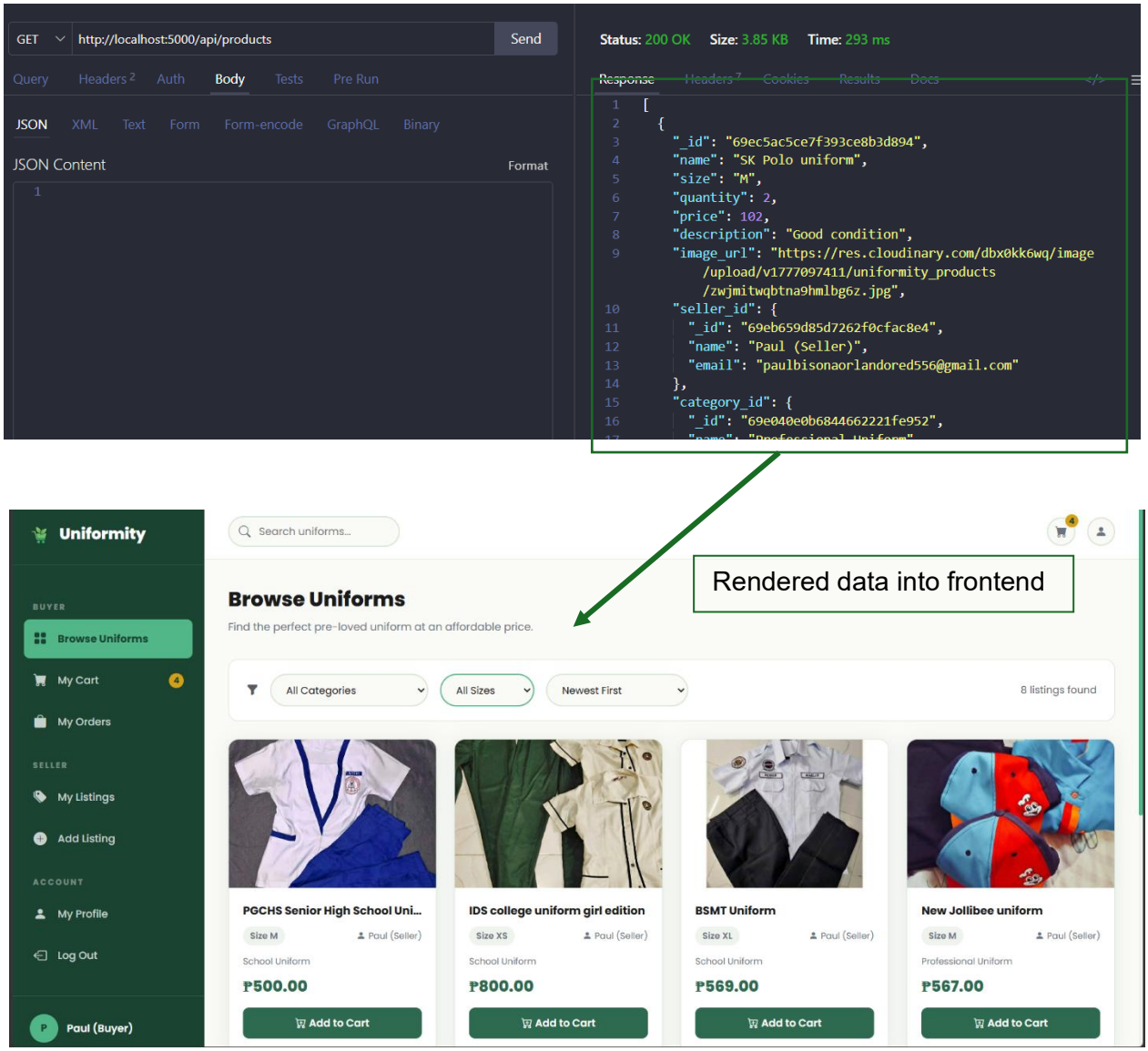
Route: GET /api/cart/:userId | Fetches and renders user cart items with quantity controls and a dynamic total.



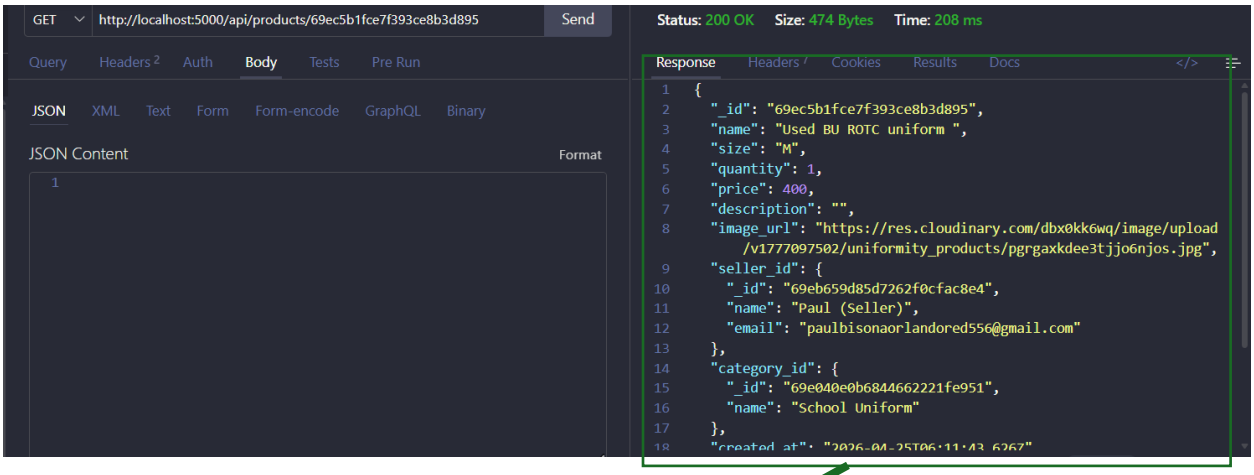
IV. Sample Backend API Responses

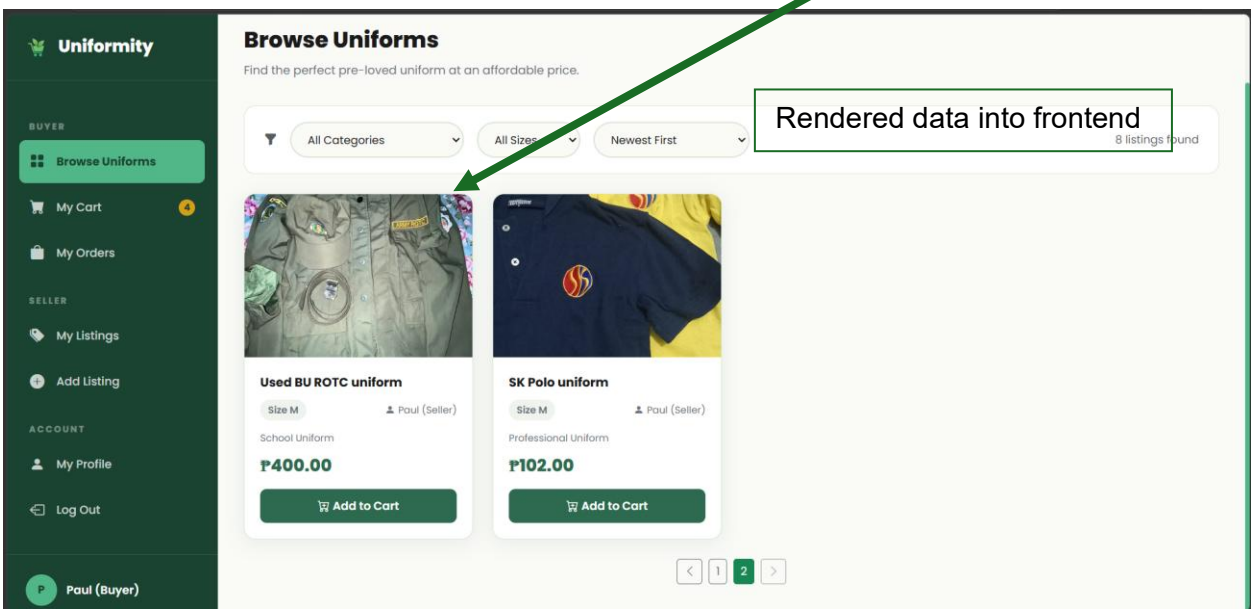
The following responses were captured using Postman and the browser console. They confirm the backend correctly serves the data that Phase 5 renders on screen.

A. GET /api/products — Product Listing Response

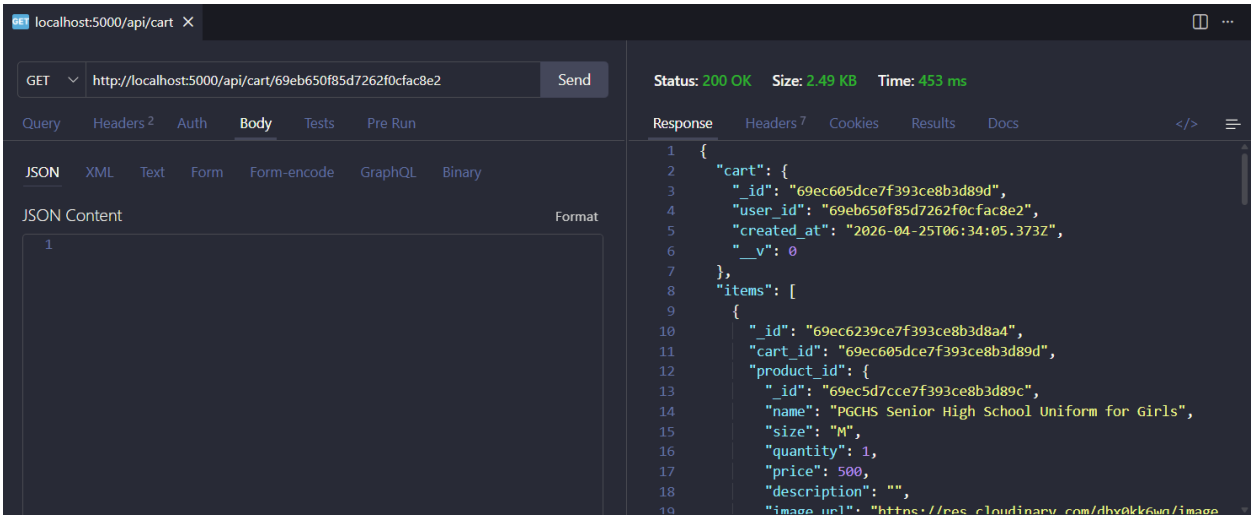


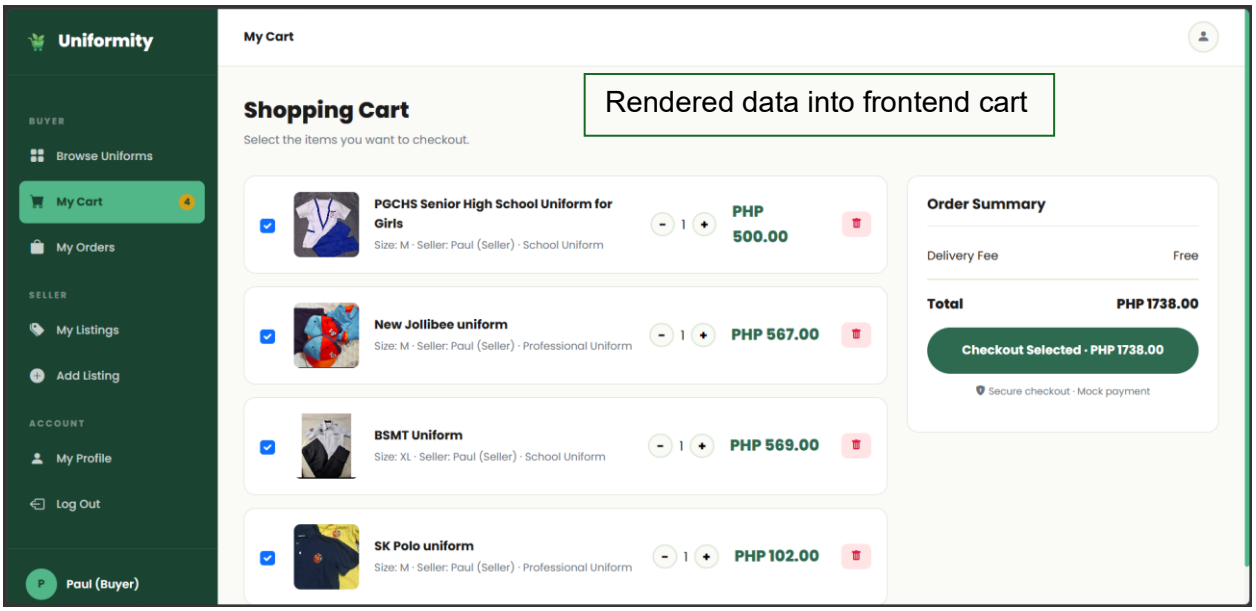
B. GET /api/products/:id — Single Product Response





C. GET /api/cart/:userId — Cart Response





V. Annotated JavaScript code Snippet in Rendering

The snippets below highlight the key fetch() and DOM manipulation logic used in Phase 5 to retrieve and render data from the backend API.

A. Product Grid Rendering — dashboard.html

```javascript
// ── Fetch All Products ──────────────────────
async function loadProducts() {
    const grid = document.getElementById('productGrid');
    grid.innerHTML = `
        <div class="col-12 text-center py-5">
            <div class="spinner-border text-success" role="status"></div>
            <p class="mt-3 text-muted" style="font-size:.9rem;">Loading uniforms...</p>
        </div>`;

    try {
        const res  = await fetch(`${API_BASE}/products`);
        const data = await res.json();
        allProducts = data;
        applyFiltersAndRender();
    } catch {
        grid.innerHTML = `
            <div class="col-12 text-center py-5">
                <i class="bi bi-wifi-off fs-1 text-muted"></i>
                <p class="mt-3 text-muted">Could not load products. Make sure the server is running.</p>
            </div>`;
    }
}
```

```
// ── Product Card Template ──────────────────
function buildProductCard(p) {
    const price     = `₱${Number(p.price).toFixed(2)}`;
    const size      = p.size || '–';
    const seller    = p.seller_id?.name || 'Seller';
    const category  = p.category_id?.name || '';
    const imgHTML   = p.image_url
        ? `<img src="${p.image_url}" alt="${p.name}" style="width:100%;height:100%;object-fit:cover;" />`
        : `<div class="product-img-placeholder"><i class="bi bi-image fs-1"></i></div>`;

    return `
        <div class="product-card" onclick="window.location.href='product-detail.html?id=${p._id}'">
            <div class="product-img">${imgHTML}</div>
            <div class="product-body">
                <div class="product-name">${p.name}</div>
                <div class="product-meta">
                    <span class="product-size">Size ${size}</span>
                    <span class="product-seller"><i class="bi bi-person-fill me-1"></i>${seller}</span>
                </div>
                <div style="font-size:.75rem;color:var(--text-muted);margin-bottom:.5rem;">${category}</div>
                <div class="product-price">${price}</div>
                <button class="btn-add-cart" onclick="event.stopPropagation(); handleAddToCart('${p._id}', '${p.seller_id?._id || ''}', ${p.price
                    <i class="bi bi-cart-plus me-1"></i>Add to Cart
                </button>
```

Uses the **Fetch API with the JWT token** to retrieve products from the backend. A loading spinner is shown while the request is pending. Each product is mapped into a Bootstrap card element with a Cloudinary image, category badge, price, and an Add to Cart button wired to the addToCart() function from api.js. Clicking the image navigates to the product detail page, passing the product ID as a URL query param.

B. Product Detail Rendering — product-detail.html

Read ?id= from URL, fetch single product, populate DOM elements

```
async function loadProductDetail() {
    const params = new URLSearchParams(window.location.search);
    const productId = params.get('id');

    if (!productId) {
        showError();
        return;
    }
```

```
function renderProduct(product) {
    const price = `PHP ${Number(product.price).toFixed(2)}`;
    const seller = product.seller_id?.name || 'Unknown Seller';
    const email = product.seller_id?.email || '';
    const category = product.category_id?.name || '-';
    const listedAt = product.createdAt || product.created_at;
    const listed = listedAt
        ? new Date(listedAt).toLocaleDateString('en-PH', { dateStyle: 'medium' })
        : '';
```

```
const imgContainer = document.getElementById('detailImage');
if (imgContainer) {
    imgContainer.innerHTML = product.image_url
        ? `<img src="${product.image_url}" alt="${product.name}" style="width:100%;border-radius:var(-
        : `<i class="bi bi-image" style="color:var(--text-muted);font-size:3rem;"></i>`;
}

setText('detailName', product.name);
setText('detailPrice', price);
setText('detailSize', product.size || '-');
setText('detailQty', product.quantity ?? '-');
setText('detailCategory', category);
setText('detailDescription', product.description || 'No description provided.');
setText('sellerName', seller);
setText('sellerEmail', email);
setText('detailListedDate', listed ? `Listed on ${listed}` : '');
```

Extracts the **product ID** from the URL query string and calls the single-product endpoint. The loading spinner is hidden and #detailBody shown only after the data arrives. All detail fields (name, price, size, category, description, seller info, image) are injected into the pre-existing DOM placeholders. The Add to Cart button is wired dynamically with the real product ID, seller ID, and price after fetch completes.

C. Buyer Orders Rendering — my-orders.html

Fetch buyer's order history and render order cards

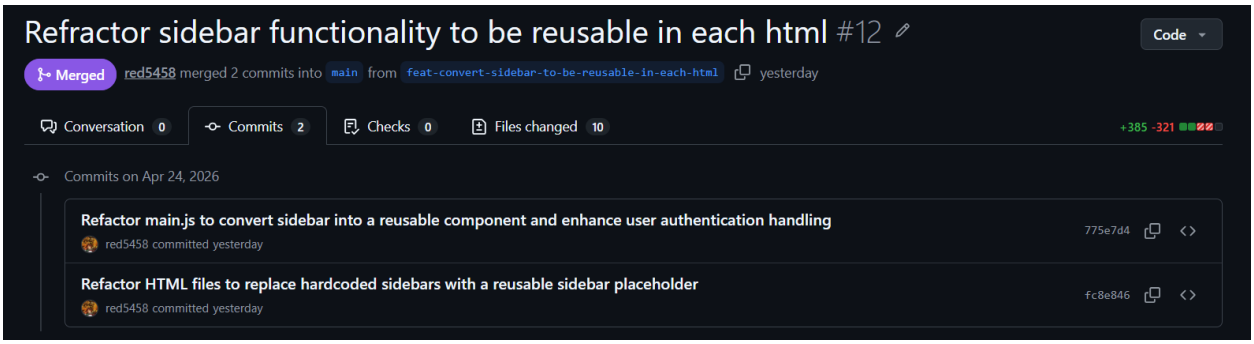
```
async function loadMyOrders() {
    const token = localStorage.getItem('token');
    const user = getUser();
    const container = document.getElementById('ordersContainer');

    if (!container || !token || !user?._id) return;

    container.innerHTML = `
        <div class="text-center py-5">
            <div class="spinner-border text-success" role="status"></div>
            <p class="mt-3 text-muted" style="font-size:.9rem;">Loading your orders...</p>
        </div>`;

    try {
        const res = await fetch(`${API_BASE}/orders`, {
            headers: { 'x-auth-token': token }
        });
        const orders = await res.json();
```

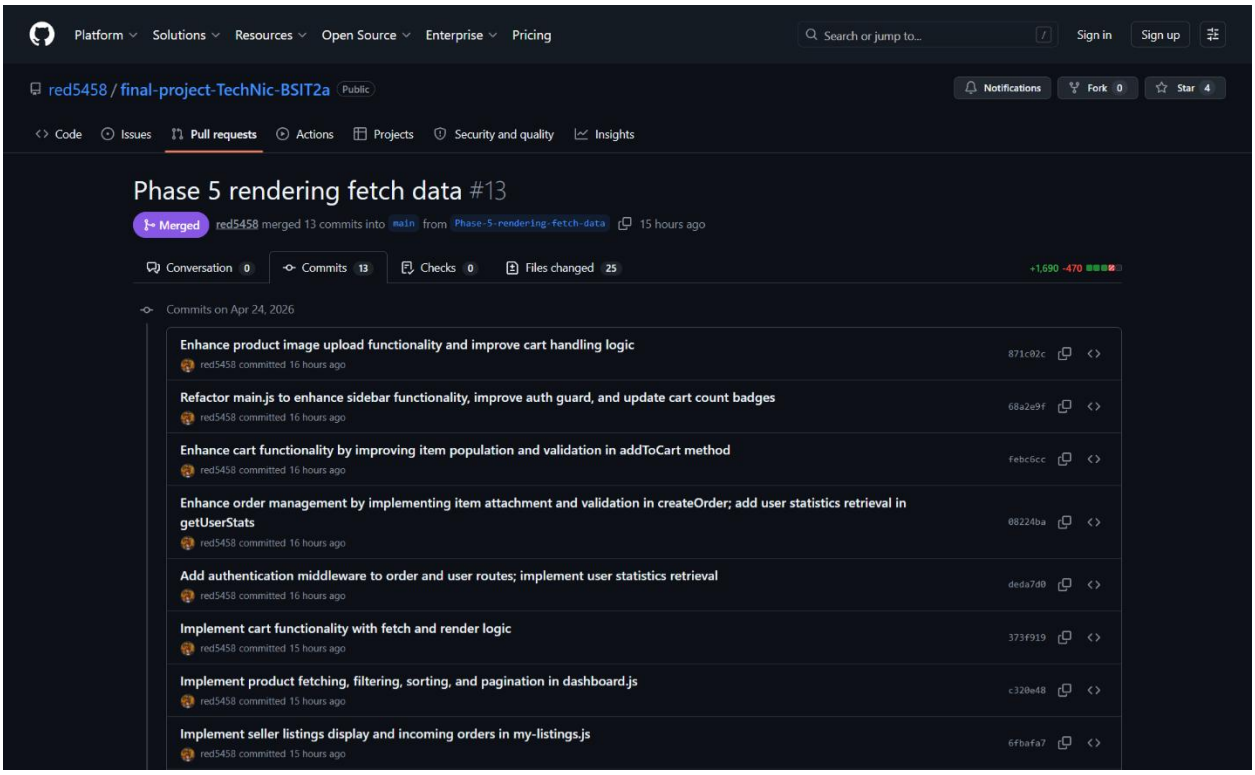
```
function renderOrders(orders) {
    const container = document.getElementById('ordersContainer');
    if (!container) return;

    container.innerHTML = `<div class="row m-0">${orders.map((order) => buildOrderCard(order)).join('')}</div>`;
}

function buildOrderCard(order) {
    const firstItem = order.items?.[0];
    const firstName = firstItem?.product_id?.name || 'Order Item';
    const extraCount = (order.items?.length || 1) - 1;
    const extraText = extraCount > 0
        ? ` <span style="font-weight:400;font-size:.85rem;color:var(--text-muted);">(+${extraCount} other item/s)</span>`
        : '';
    const total = `PHP ${Number(order.total_amount).toFixed(2)}`;
    const orderDate = order.createdAt || order.created_at;
    const placedDate = orderDate
        ? new Date(orderDate).toLocaleDateString('en-PH', { dateStyle: 'medium' })
        : '-';
```

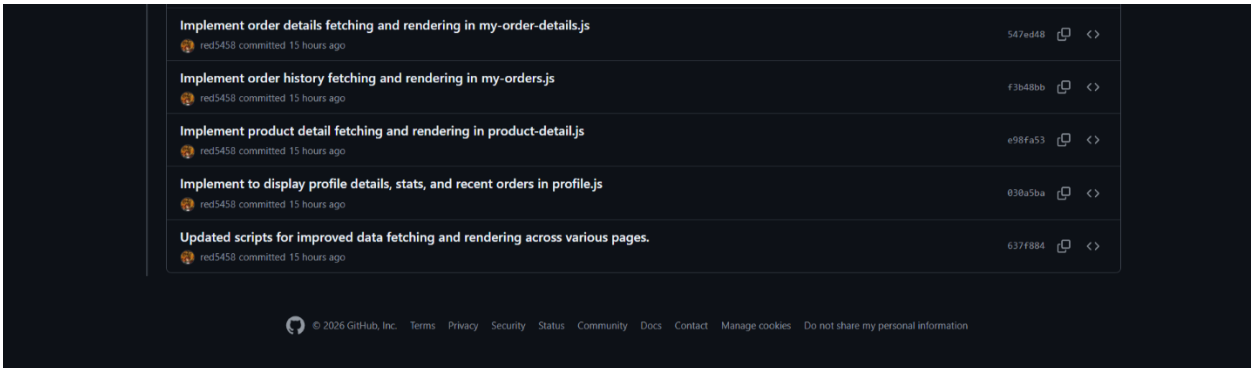
Retrieves the logged-in user's ID from localStorage (saved during login in Phase 4) and fetches all their orders from the backend. Each order is rendered as a clickable card with a truncated order ID, color-coded status badge using the existing CSS classes, total amount, and formatted date. Clicking any card navigates to the detailed order view, passing the order ID as a URL parameter.

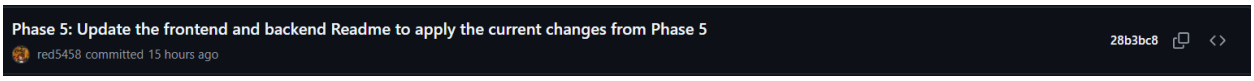
VI. GitHub Commit Logs

The following commits represent the Phase 5 implementation, organized under a dedicated branch and merged into main via pull request.









VII. Short Reflections

Cipriano, Mcxyron B. — Back-End Developer

In this Phase 5, I actually learned how the populate() method I used in the product and order controllers made the frontend rendering much simpler, since it eliminated extra API calls by embedding seller and category data directly in the response. I worked closely with our frontend developer to ensure the API responses is correct for rendering, tweak some of the backend codes to align with our data retrieval. Seeing the product cards appear in real time with our actual MongoDB data was the most satisfying moment of the entire project for me.

Hernandez, Kurt Jushua S. — Database Manager / Tester & Debugger

I focused on testing every data display scenario including empty states, invalid product IDs, and expired tokens to make sure the frontend handled errors. One key thing I verified was that MongoDB ObjectId references were correctly resolved by Mongoose populate(), which was essential for the seller card and category badge to display properly. This phase showed me how thorough testing of data flow is just as important as making the code itself.

Registrado, Mharie Franz — Tester & Debugger

Testing Phase 5 was more involved because the displayed data depends on what is actually in the database, so I had to ensure test category data was properly seeded before validating UI output. I caught a few cases where the product image would break if image_url was an empty string rather than null, and reported this to our manager so a error placeholder icon was added. This phase taught me to think about edge cases in data-driven UIs, not just form validation.

Red, Paul Orlando B. — Project, Database, GitHub manager & Documentation Officer

Managing Phase 5 required coordinating the exact data the frontend expected with what the backend was returning — I updated the documentation as features were completed and organized the PR to keep the commit history clean and readable. I also ensured our MongoDB Atlas had enough realistic seed data for screenshots to be meaningful and also helps in making the overall codes for whole phase 5. Writing this lab report helped me trace the complete data journey from a MongoDB document, through an Express route, to a rendered Bootstrap card, which gave me the clearest picture yet of how the full stack fits together.

Romano, Jay L. — Front-End Developer

Phase 5 was the most complex frontend work I have done so far because I had to manage asynchronous fetch calls, spinner states, error states, and DOM injection all at once without a framework. Our project manager help me in implementation for this phase and learned how to use URLSearchParams to read query parameters for the product detail page, and how to use event delegation for the quantity buttons in the cart so they work on dynamically created elements. Seeing the full app go from hardcoded HTML to a fully data-driven interface help me realize that through every process, we can obtain experiences that will definitely benefit me in my future web projects.

AI usage: *Around 10% of AI-generated content was used in this lab report to help me, as the Documentation Officer, structure the document and guide what specific screenshots to include along with proper annotations. However, all the information presented in this documentation is based on our actual work, outputs, and understanding of the project. The content reflects our team's efforts, testing, and implementation throughout each phase.*